

Ficha prática #3 (Resolução)

Programação em *Shell* III

1. Faça um comando **datahoras**, usando o comando *date*, que imprima o dia da semana em português.

```
#!/bin/bash
set `date`
case $1 in
    Sun) dsem="domingo";;
    Mon) dsem="segunda-feira";;
    Tue) dsem="terça-feira";;
    Wed) dsem="quarta-feira";;
    Thu) dsem="quinta-feira";;
    Fri) dsem="sexta-feira";;
    Sat) dsem="sábado";;
esac

data=`echo $3/$2/$6, $dsem`
IFS=:      #IFS-Internal Field Separator é uma variável da shell
set $4
horas=`echo ${1}H$2`
echo "São $horas do dia $data"
```

2. Faça o comando **vib** que edite um ficheiro mantendo uma cópia da penúltima versão do ficheiro editado, com a extensão *.bak*. O comando não deve editar ficheiros com a extensão *.bak*.

```
#!/bin/bash
case $# in
    0) echo "Uso: vib ficheiro" 1>&2; exit 1;;
esac
case $1 in
    *.bak) echo "Não é permitido editar ficheiros com a extensão
bak" 1>&2; exit 1;;
esac

file=/tmp/$1.$$ #$$ é o Process ID (PID)
if test -f $1
then cp $1 $file
    gedit $file      # ou vim, ou vi, qualquer editor de texto
    if cmp $file $1 1>/dev/null 2>&1
    then rm $file
    else mv $1 $1.bak
        mv $file $1
    fi
else gedit $1
fi

# 0 - Ficheiro de entrada, que é por defeito o teclado
# 1 - Ficheiro de saída, que é por defeito o terminal
# 2 - Ficheiro de saída de erro, que é por defeito o terminal
# A notação 2>&1 indica à shell para enviar a saída de erro
# (descriptor 2) para o ficheiro de saída (descriptor 1). O inverso
# 1>&2 também é válido quando se pretende enviar mensagens de erro.
# O operador & permite juntar a saída do erro com a saída, no mesmo
```



ficheiro.

3. Faça um comando **dlist** que faça a listagem da estrutura de diretórios a partir do diretório dado pelo primeiro parâmetro de entrada

```
#!/bin/bash
case $# in
  0) echo "Uso: dlist directório" 1>&2; exit 1;;
  *) if test ! -d $1
      then echo "$1 não é um directório" 1>&2
        exit 1
      fi;;
esac

cd $1
for i in `ls`
do
  if test -d $i
  then dlist $i
  fi
done
```

4. Faça um comando **backup** que cria uma cópia de proteção do diretório passado como segundo parâmetro, no diretório dado pelo primeiro parâmetro. Para tornar a execução do comando mais eficiente, apenas são copiados os ficheiros que sofreram alteração desde a última vez que o comando foi executado, incluindo os ficheiros novos. Se o diretório de backup não existir, deverá ser criado.

```
#!/bin/bash
case $# in
  *|1) echo "Uso: backup dir_back dir_orig" 1>&2; exit 1;;
esac
test -d $1 || mkdir $1
if test ! -d $2
then echo "$2 não é um directório" 1>&2; exit 1;;
fi
for i in `ls $2`
do
  if test -f $1/$i
  then if cmp $1/$i $2/$i >/dev/null 2>&1
        then :
        else cp $2/$i $1/$i
        fi
  else cp $2/$i $1/$i
  fi
done
```



5. Faça um comando **rfd** que apaga todos os ficheiros com nome igual ao segundo parâmetro, que existam na estrutura de directórios a partir do directório indicado como primeiro parâmetro. O comando deve mostrar o *pathname* do ficheiro e perguntar ao utilizador se quer mesmo apagá-lo, com uma opção de abandono (*quit*).

```
#!/bin/bash
case $# in
  0|1) echo "Uso: rfd directório ficheiro" 1>&2; exit 1;;
esac

if test ! -d $1
then echo "$1 não é um directório" 1>&2; exit 1;;
fi

cd $1
for i in `ls`
do
  if test -d $i
  then rfd $i $2
  else
    if ( test -f $i ) && ( test $i = $2 )
    then echo -n "Quer mesmo apagar `pwd`/$i [s/n/q]?
      read resposta
      case $resposta in
        s*|S*) rm $i;;
        q*|Q*) break ;;
      esac
    fi
  fi
done
```